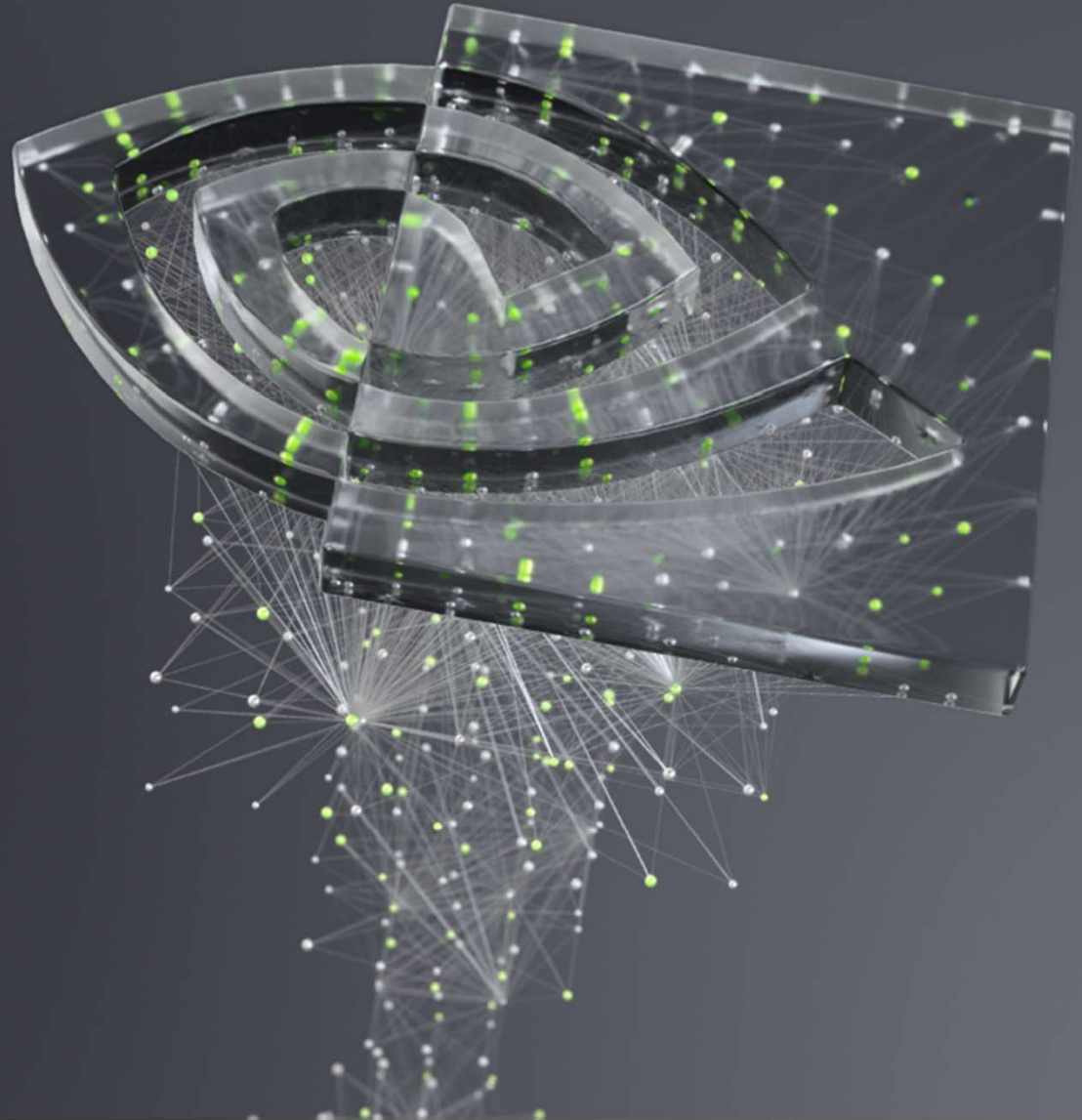


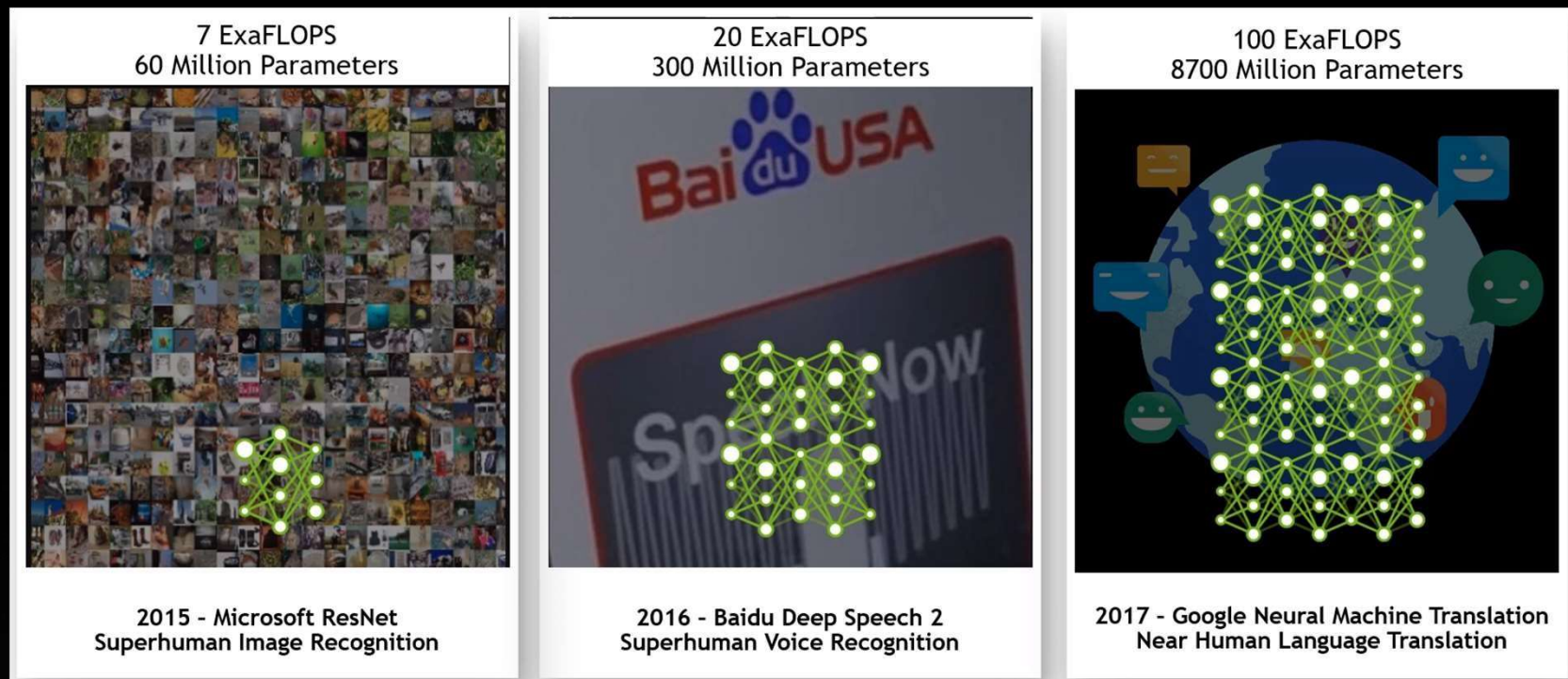


DISTRIBUTED DEEP LEARNING

End to End Accelerated Deep Learning

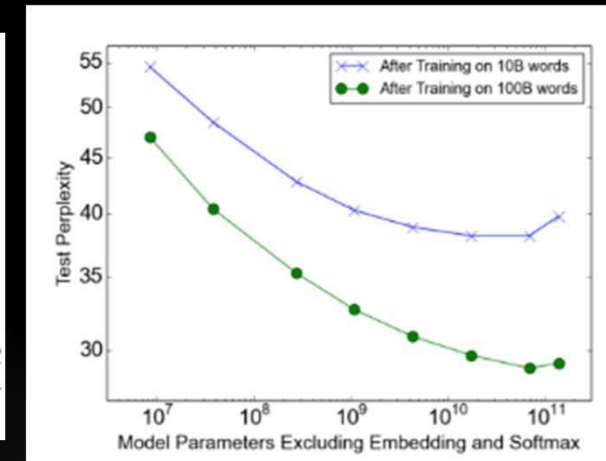
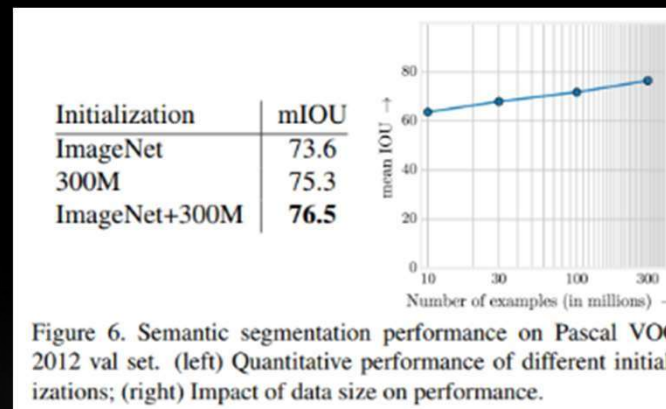
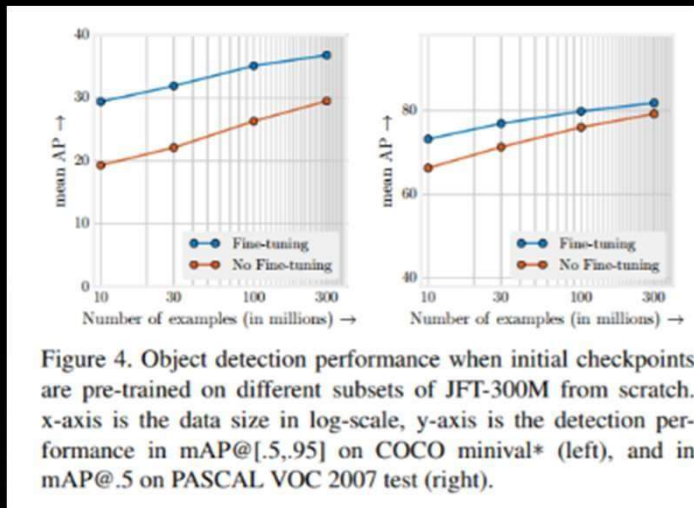


THE NEED FOR DISTRIBUTED DEEP LEARNING



Motivations for Multi-GPU Training

DL benefits from training on larger datasets

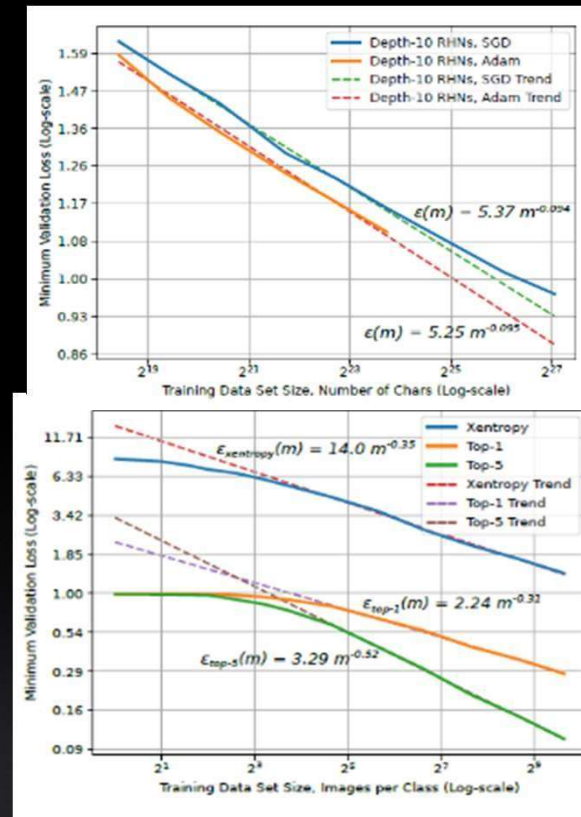
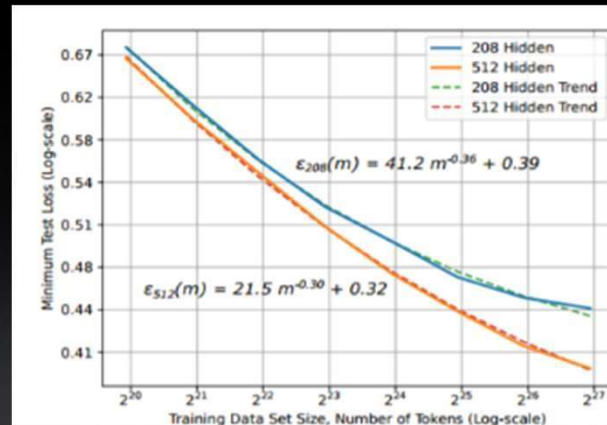
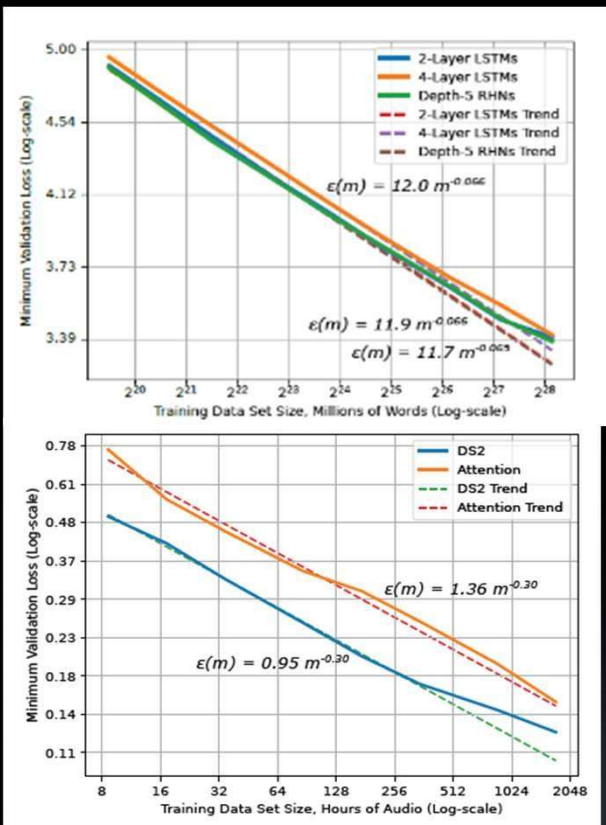


Sun, Chen, et al. "Revisiting Unreasonable Effectiveness of Data in Deep Learning Era." *arXiv preprint arXiv:1707.02968* (2017).
 Shazeef, Noam, et al. "Outrageously large neural networks: The sparsely-gated mixture-of-experts layer." *arXiv preprint arXiv:1701.06538* (2017).
 Simonyan, Karen, and Andrew Zisserman. "Very deep convolutional networks for large-scale image recognition." *arXiv preprint arXiv:1409.1556* (2014).

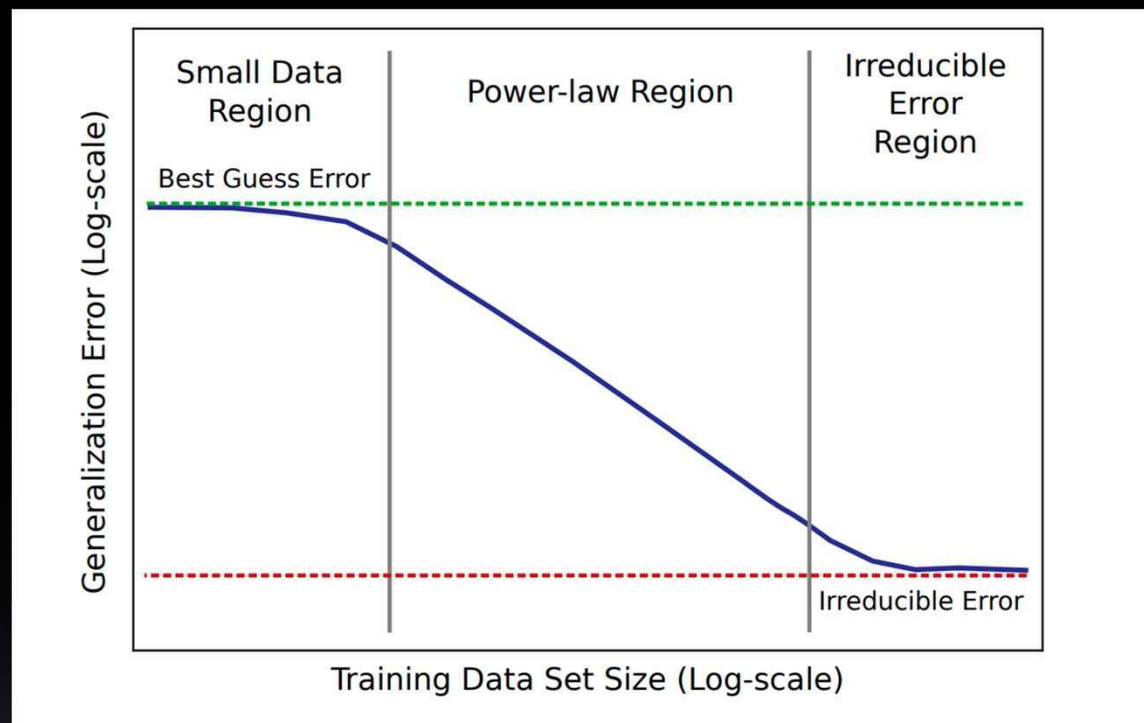
Motivations for Multi-GPU Training

DL benefits from training on larger datasets

- Translation
- Language Models
- Character Language Models
- Image Classification
- Attention Speech Models

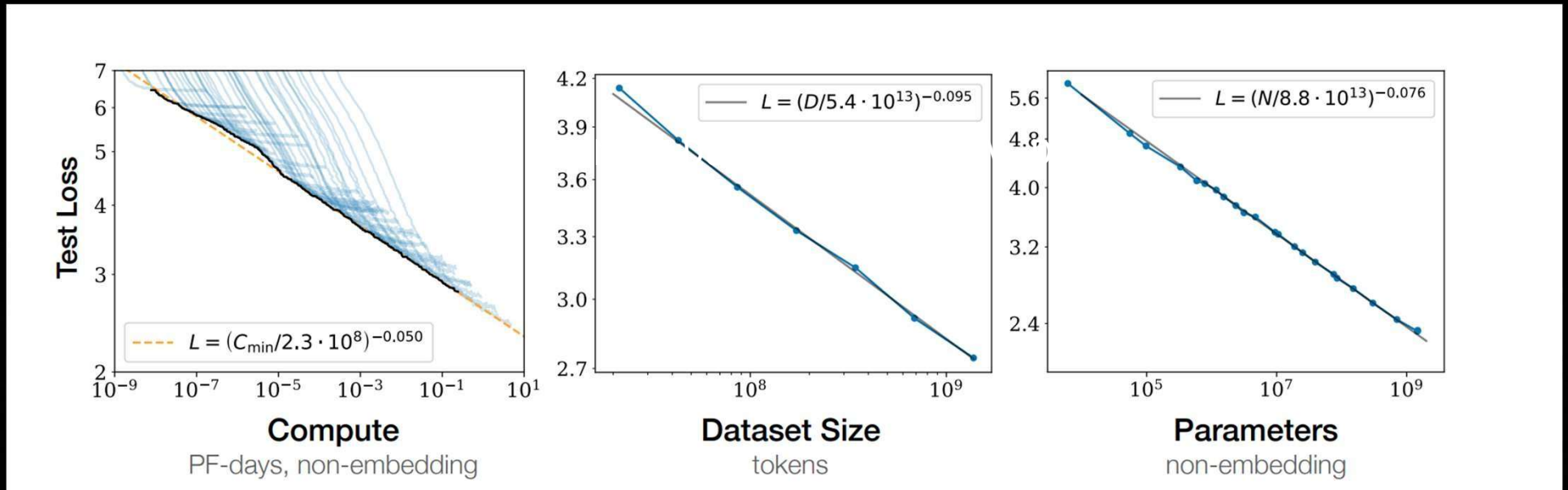


THE NEED FOR DISTRIBUTED DEEP LEARNING



THE NEED FOR DISTRIBUTED DEEP LEARNING

The Scaling laws



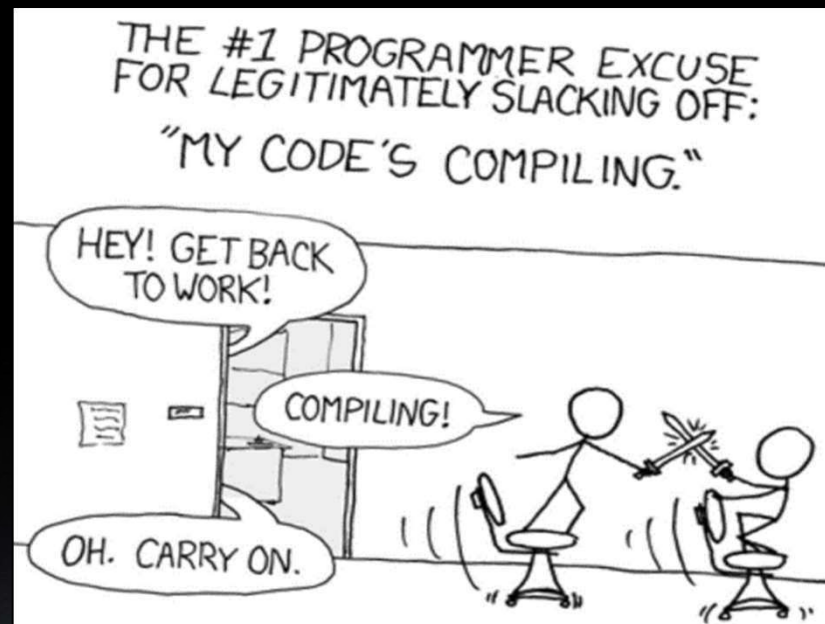
Motivations for Multi-GPU Training

Implications from needing larger data, models

- The good news: Requirements are predictable.
 - We can predict how much data we will need
 - We can predict how much computing power we will need
- The bad news: The values can be significant.

Motivations for Multi-GPU Training

Implications from needing larger data, models



Multi-GPU Training

Benefits

Within the same duration of time, we can:

- Train on more data
- Train more iterations
- Train larger models

Thus, we can get better results, or, get the same results faster.

Multi-GPU Training

Benefits

ResNet-50 (90 epochs training on ImageNet):

1x A100: 14 hours

8x A100: 3 hours

BERT-Large (pretraining on BookCorpus)

8x V100: 153 hours

16x V100: 58 hours

1024x V100: 1 hour

We can reduce the time it takes to get results by using more GPUs.

Multi-GPU Training

Overview of Methods

We have two main methods of spreading our training workload across GPUs:

Spread the input data across GPUs

This is known as **data parallelism**

This is the most straightforward way to increase training throughput

Spread the model computation across GPUs

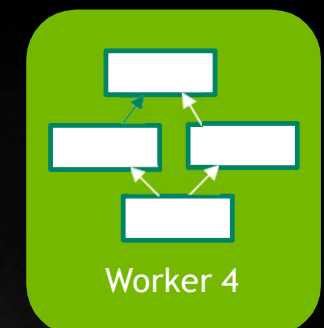
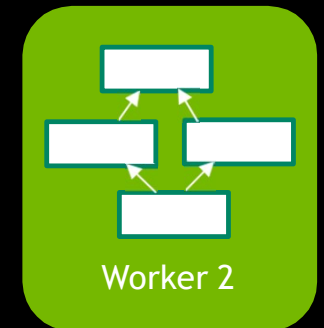
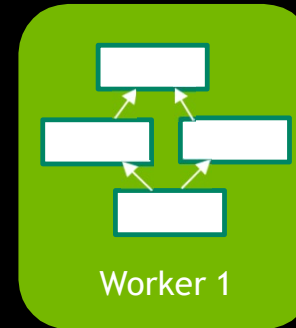
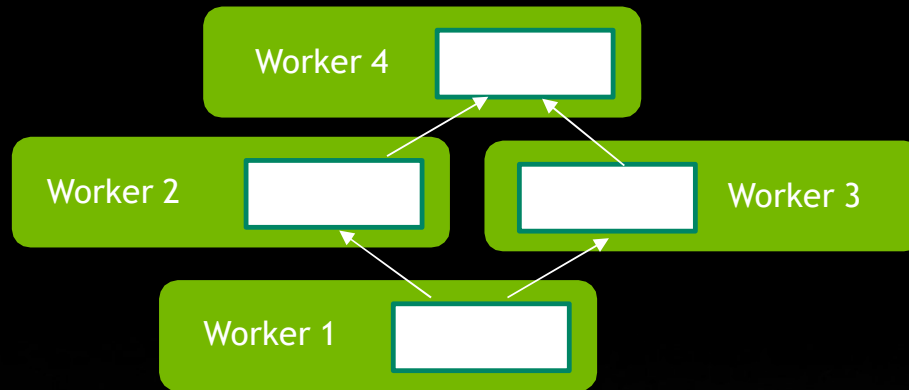
This is known as **model parallelism**

This is more complicated and has various different methods to implement

(E.g. different layers on different GPU, shard single layers onto multiple GPUs etc.)

TYPES OF PARALLELISM

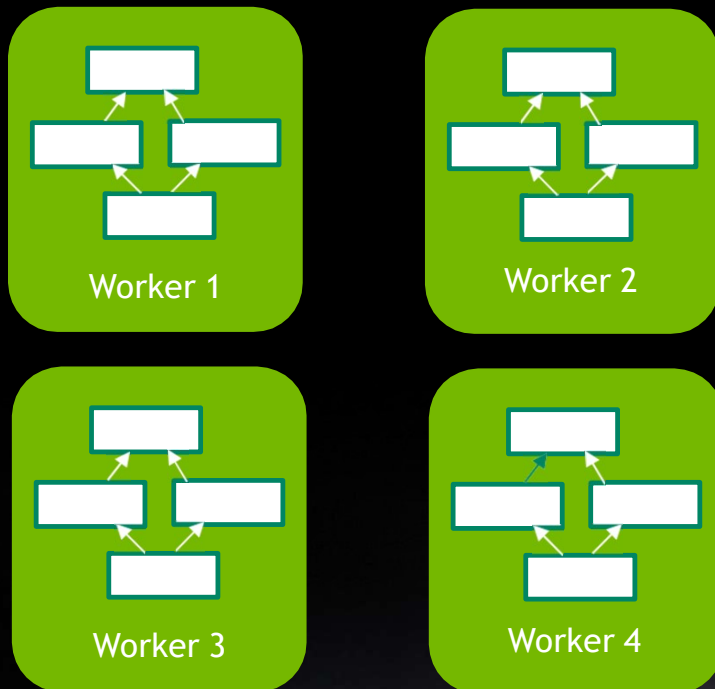
Model vs Data Parallelism



Layer

TYPES OF PARALLELISM

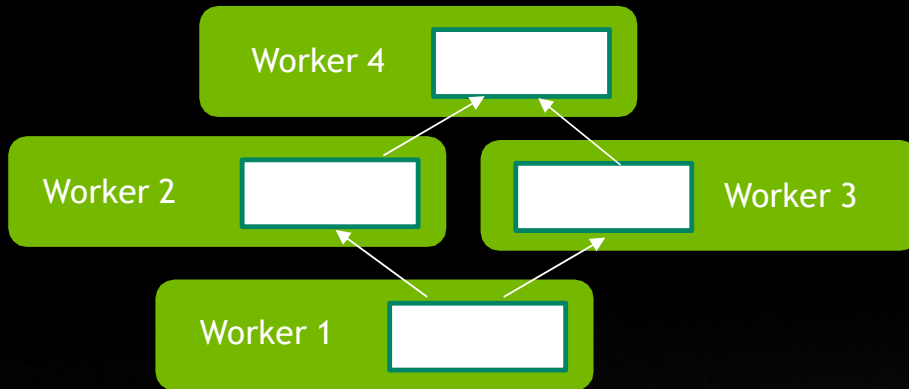
Data Parallelism



- **Data Parallelism**
 - Speed up training
 - All workers train on different data
 - All workers have same copy of model
 - Gradients (weight changes) are exchanged during training

TYPES OF PARALLELISM

Model Parallelism



- **Model Parallelism**

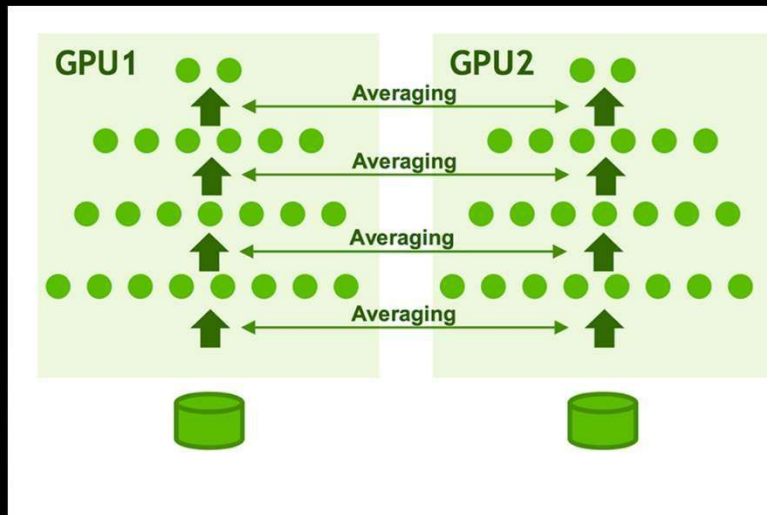
- Allows usage of bigger model (that cannot fit on a single GPU)
- All workers train on same data
- Parts of model are distributed across GPUs
- Activations (layer outputs) are passed between GPUs
- Much harder to implement in practice

Layer

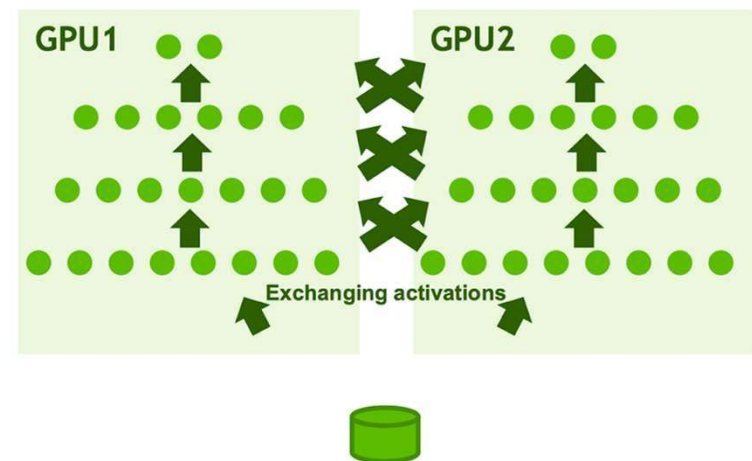
Data vs Model Parallelism

Comparison of Methods

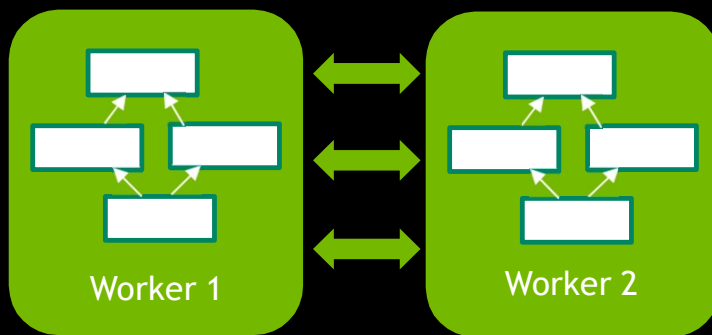
• Data Parallelism



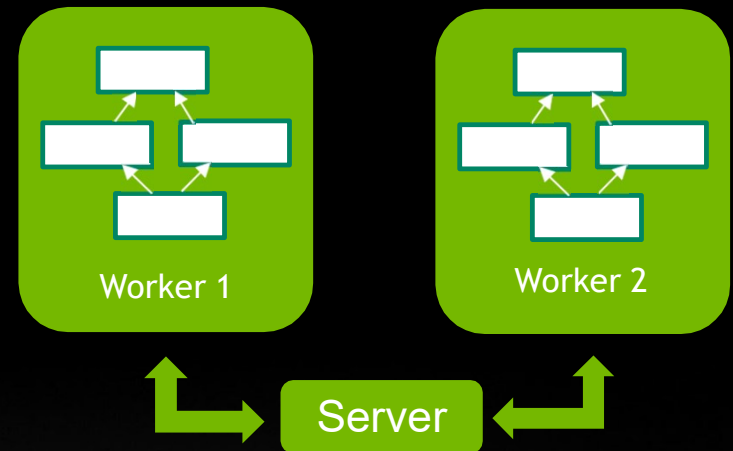
• Model Parallelism



Synchronous vs Asynchronous



Each variable in the model is mirrored across all the replicas and these variables are kept in sync with each other by applying identical updates.



Variables are created on parameter servers and workers read and update these variables independently without synchronizing with each other.

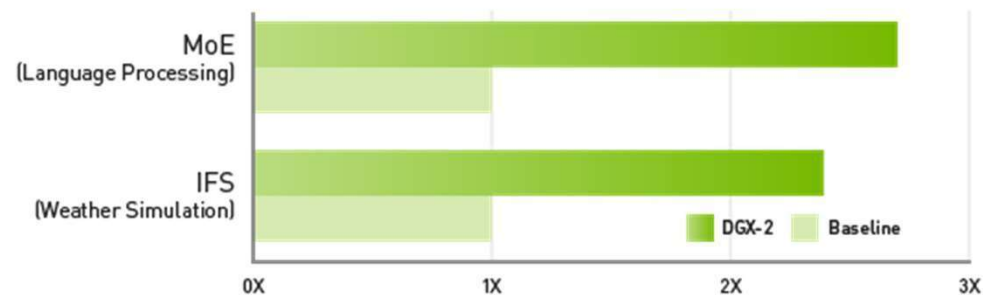


SYSTEM TOPOLOGY

SYSTEM TOPOLOGY

Importance of System Topology

NVSwitch Delivers a >2X Speedup for Deep Learning and HPC*

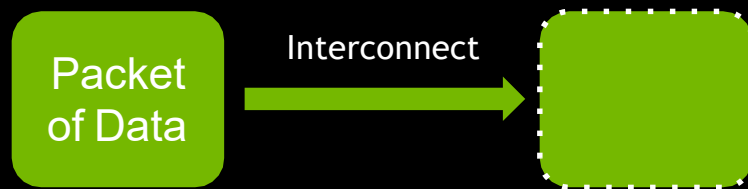


System Configs: Each of the two DGX-1 servers have dual-socket Xeon E5 2690v4 Processor, 8 x V100 GPUs; servers connected via a 4 EDR (100Gb) InfiniBand connections. DGX-2 server has dual-socket Xeon Scalable Processor Platinum 8168 Processors, 16 x Tesla V100 GPUs.

<https://developer.nvidia.com/blog/nvswitch-leveraging-nvlink-to-maximum-effect/>

UNDERSTANDING SYSTEM TOPOLOGY

Latency vs Throughput



Time taken for the transfer of a single packet of data

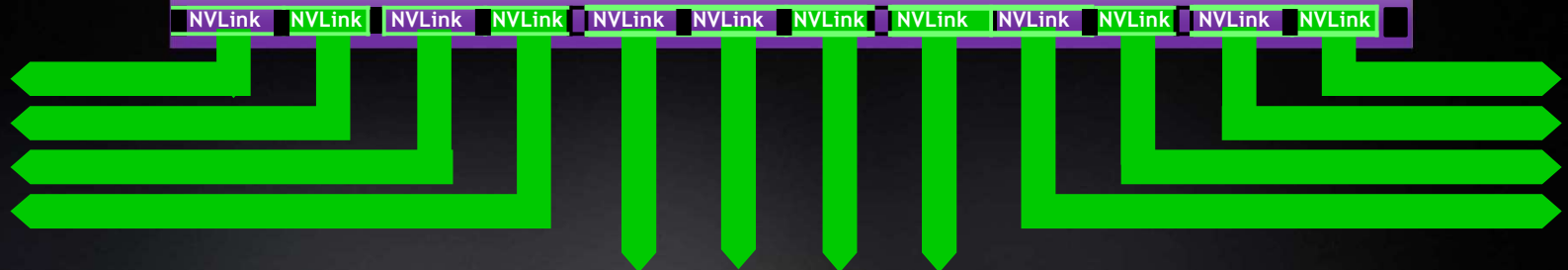
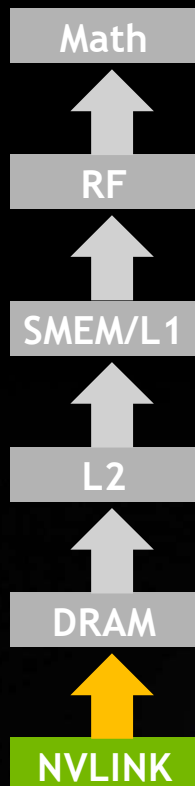
Quantity of data that can be transferred in a unit of time

$$\text{Throughput} = \text{Bus width} \div \text{Latency}$$

A100 NVLINK BANDWIDTH

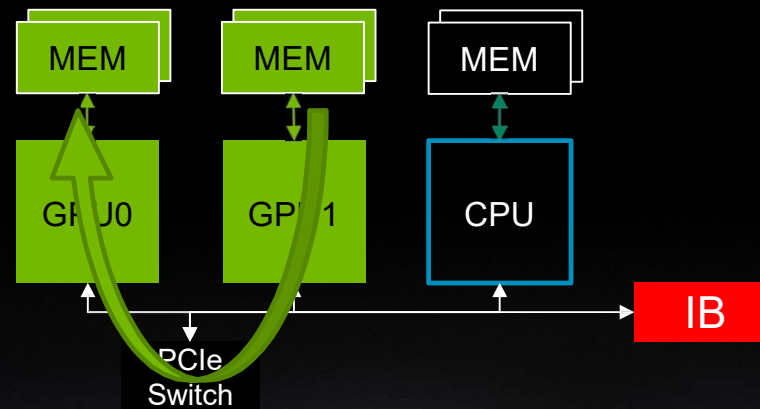
Third Generation NVLink

50 Gbit/sec per signal pair
 12 links, 25 GB/s in/out, 600 GB/s total
 2x vs. V100, 3.75x vs. P100



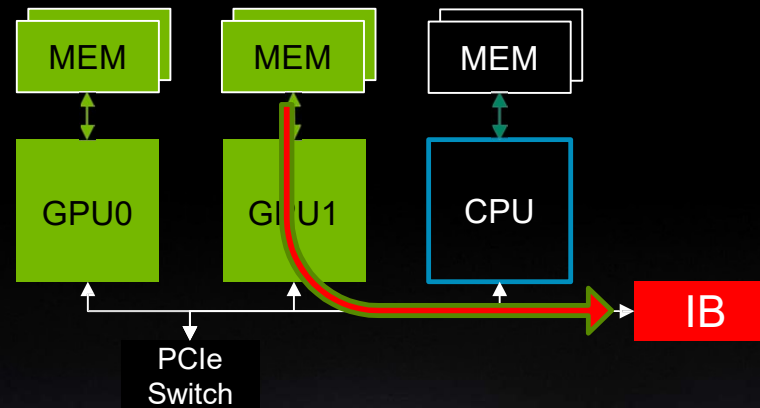
NVIDIA GPUDIRECT™

Peer to Peer Transfers



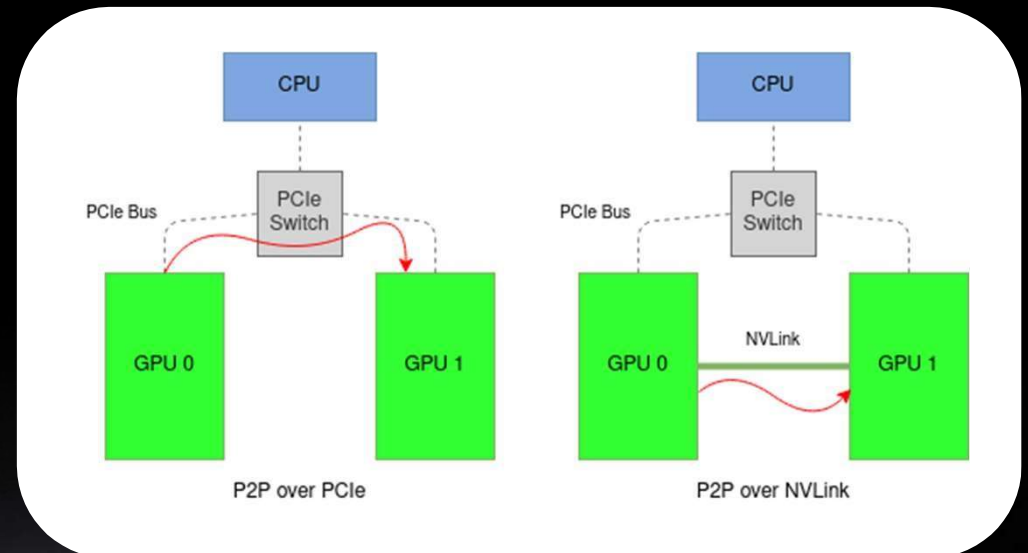
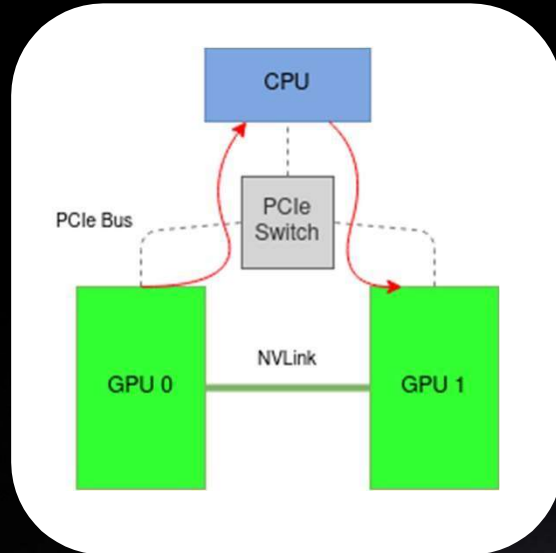
NVIDIA GPUDIRECT™

Support for RDMA



COMMUNICATION CONCEPTS

Host-Staging vs P2P Memory Access



SYSTEM TOPOLOGY

Inter-node communication topology of DGX-1

```
$ nvidia-smi topo -m
```

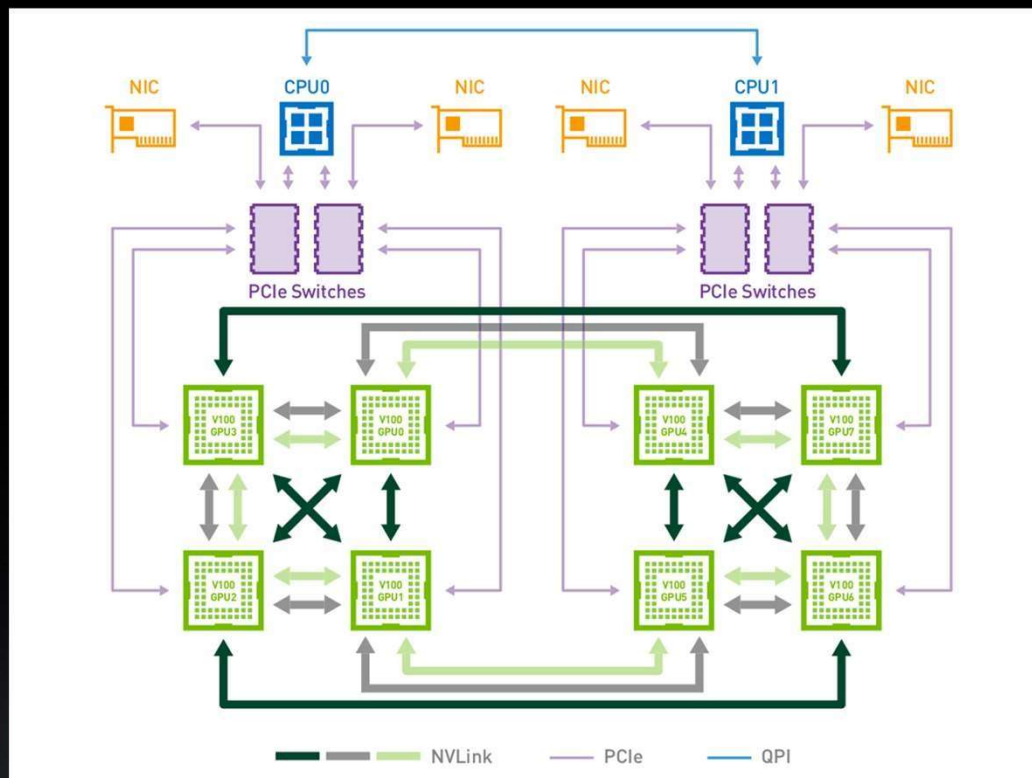
	<u>GPU0</u>	<u>GPU1</u>	<u>GPU2</u>	<u>GPU3</u>	<u>GPU4</u>	<u>GPU5</u>	<u>GPU6</u>	<u>GPU7</u>	<u>mlx5_0</u>	<u>mlx5_1</u>	<u>mlx5_2</u>	<u>mlx5_3</u>	<u>CPU Affinity</u>	<u>NUMA Affinity</u>
GPU0	X	NV1	NV1	NV2	NV2	SYS	SYS	SYS	PIX	PHB	SYS	SYS	0-19,40-59	0
GPU1	NV1	X	NV2	NV1	SYS	NV2	SYS	SYS	PIX	PHB	SYS	SYS	0-19,40-59	0
GPU2	NV1	NV2	X	NV2	SYS	SYS	NV1	SYS	PHB	PIX	SYS	SYS	0-19,40-59	0
GPU3	NV2	NV1	NV2	X	SYS	SYS	SYS	NV1	PHB	PIX	SYS	SYS	0-19,40-59	0
GPU4	NV2	SYS	SYS	SYS	X	NV1	NV1	NV2	SYS	SYS	PIX	PHB	20-39,60-79	1
GPU5	SYS	NV2	SYS	SYS	SYS	NV1	X	NV2	SYS	SYS	PIX	PHB	20-39,60-79	1
GPU6	SYS	SYS	NV1	SYS	NV1	NV2	X	NV2	SYS	SYS	PHB	PIX	20-39,60-79	1
GPU7	SYS	SYS	SYS	NV1	NV2	NV1	NV2	X	SYS	SYS	PHB	PIX	20-39,60-79	1
mlx5_0	PIX	PIX	PHB	PHB	SYS	SYS	SYS	SYS	X	PHB	SYS	SYS		
mlx5_1	PHB	PHB	PIX	PIX	SYS	SYS	SYS	SYS	PHB	X	SYS	SYS		
mlx5_2	SYS	SYS	SYS	SYS	PIX	PIX	PHB	PHB	SYS	SYS	X	PHB		
mlx5_3	SYS	SYS	SYS	SYS	PHB	PHB	PIX	PIX	SYS	SYS	PHB	X		

Legend:

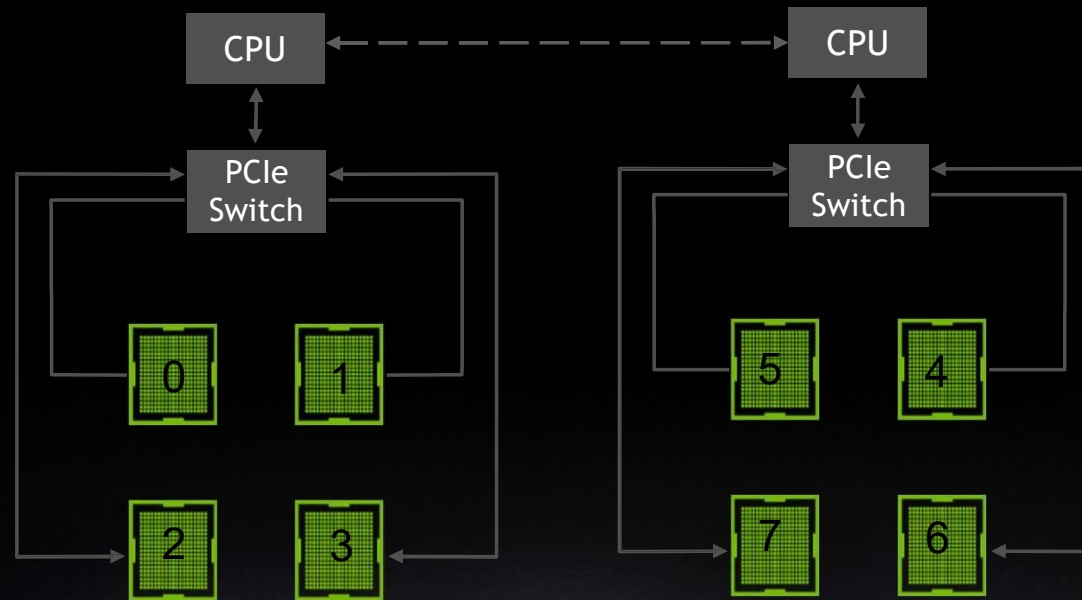
X = Self
 SYS = Connection traversing PCIe as well as the SMP interconnect between NUMA nodes (e.g., QPI/UPI)
 NODE = Connection traversing PCIe as well as the interconnect between PCIe Host Bridges within a NUMA node
 PHB = Connection traversing PCIe as well as a PCIe Host Bridge (typically the CPU)
 PXB = Connection traversing multiple PCIe bridges (without traversing the PCIe Host Bridge)
 PIX = Connection traversing at most a single PCIe bridge
 NV# = Connection traversing a bonded set of # NVLinks

SYSTEM TOPOLOGY

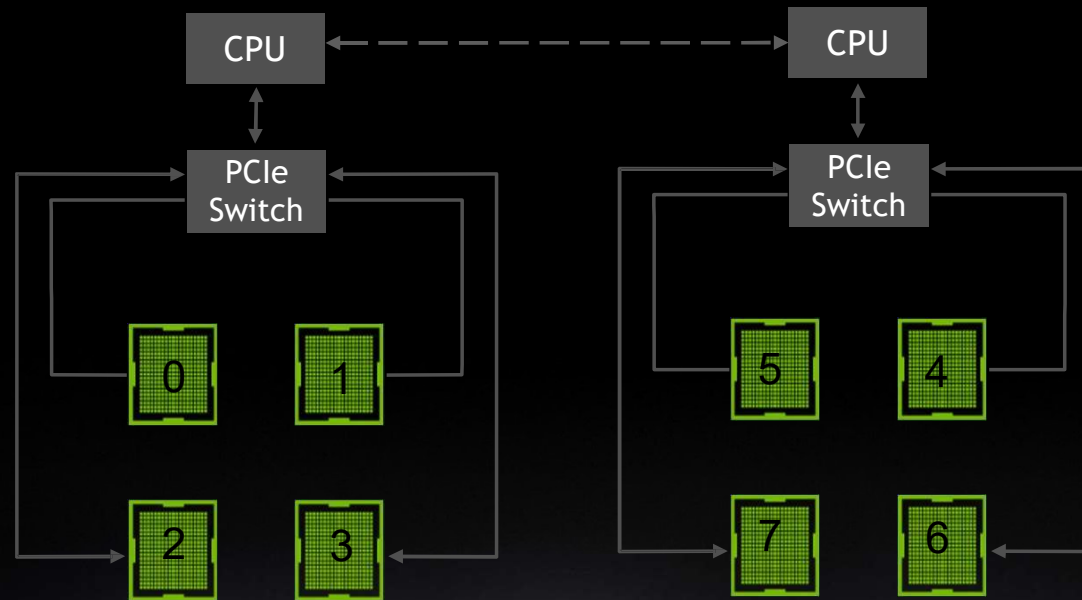
Inter-node communication topology of DGX-1



DL DATA PARALLELISM - PCIE BASED

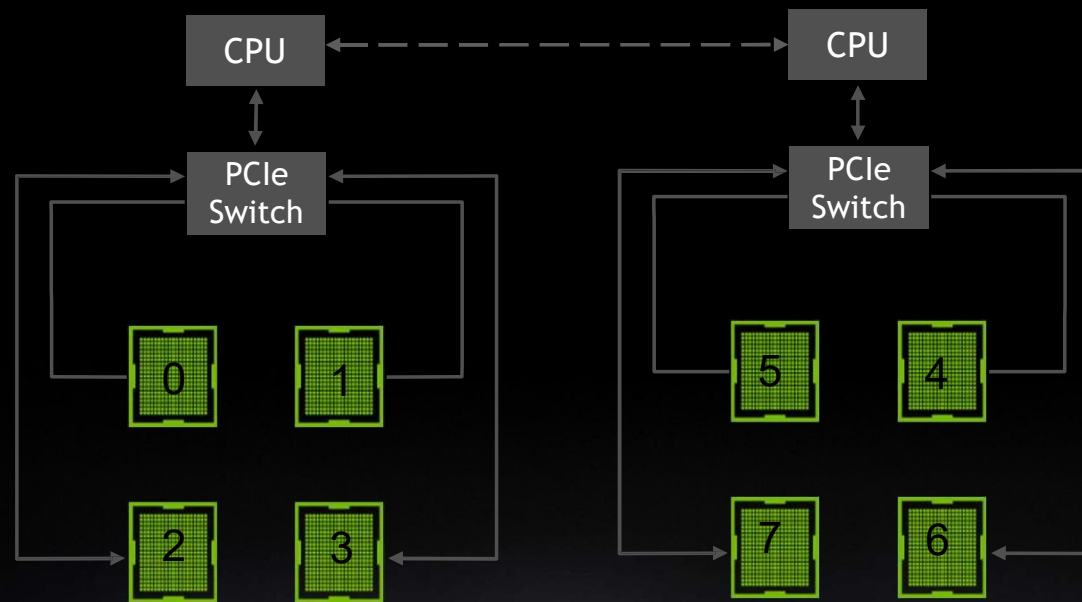


DL DATA PARALLELISM - PCIE BASED



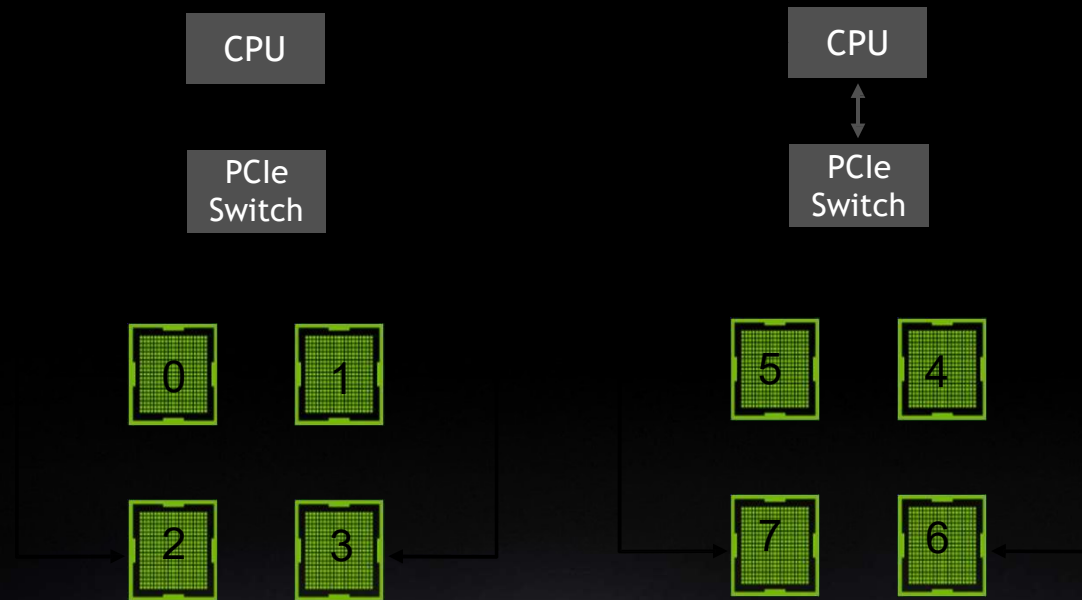
Data loading over
PCIe

DL DATA PARALLELISM - PCIE BASED



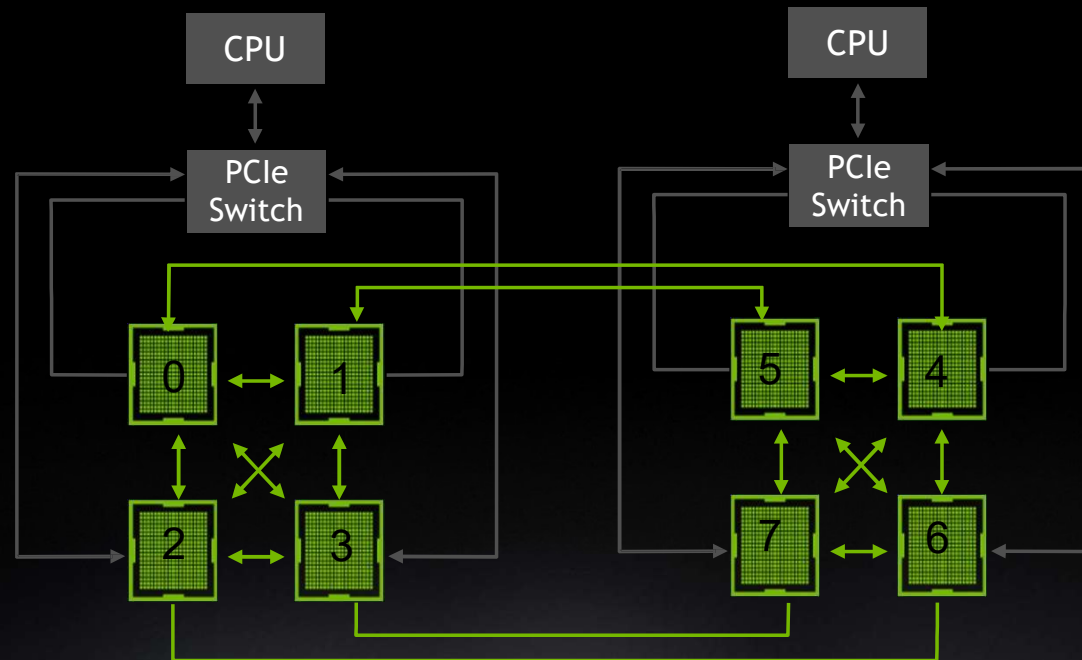
Gradient averaging over PCIe and QPI

DL DATA PARALLELISM - PCIE BASED

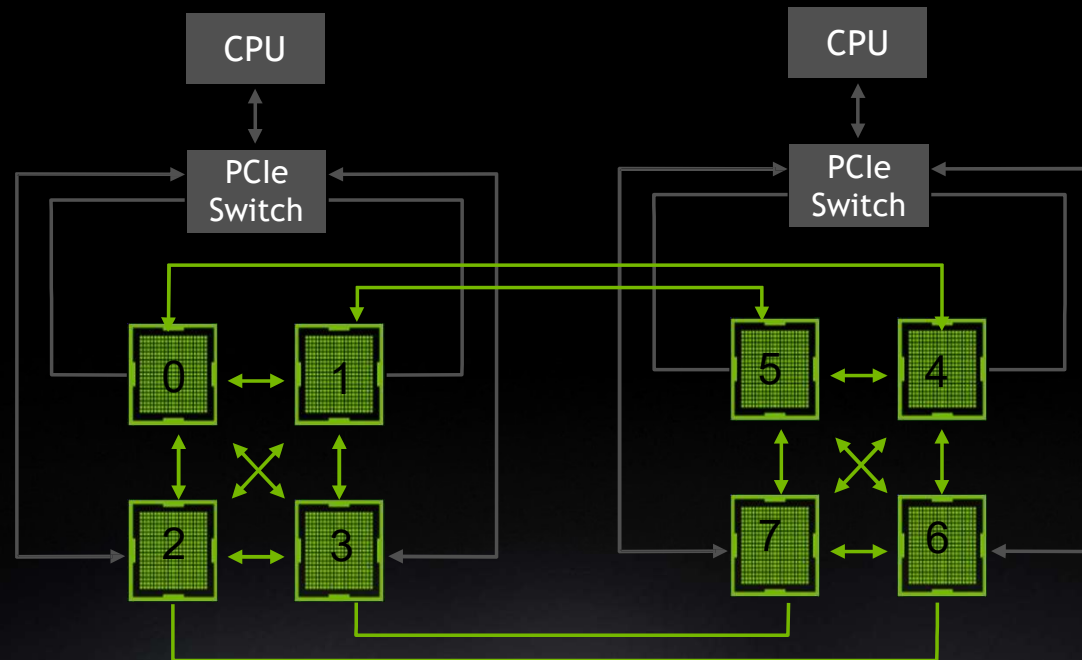


Data loading and gradient averaging share communication resources: Congestion

DL DATA PARALLELISM - NVLINK

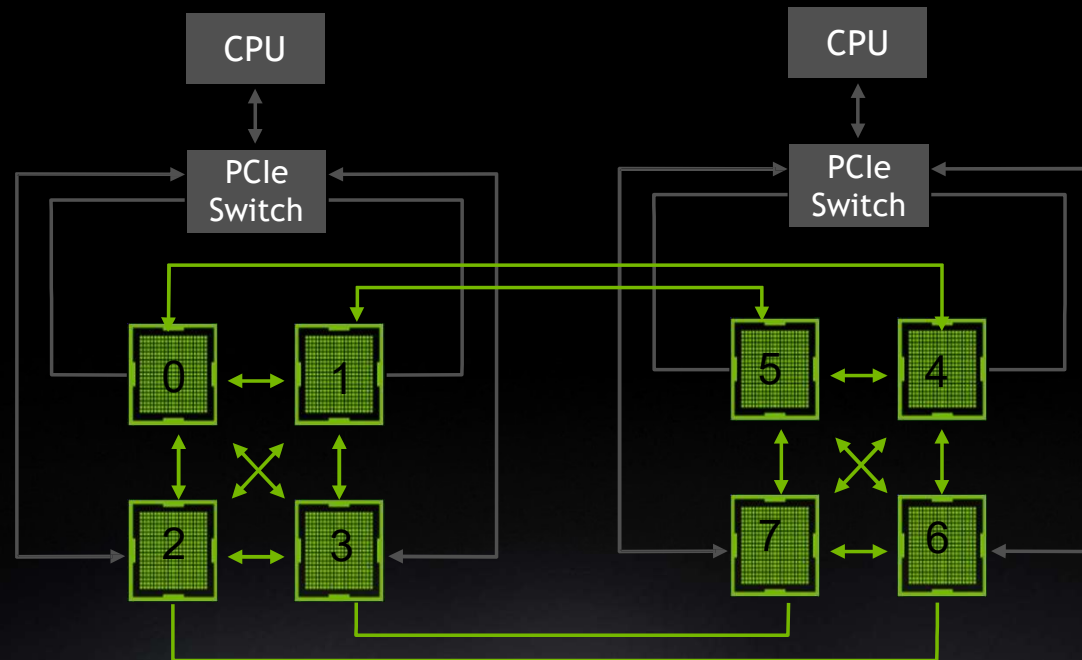


DL DATA PARALLELISM - NVLINK



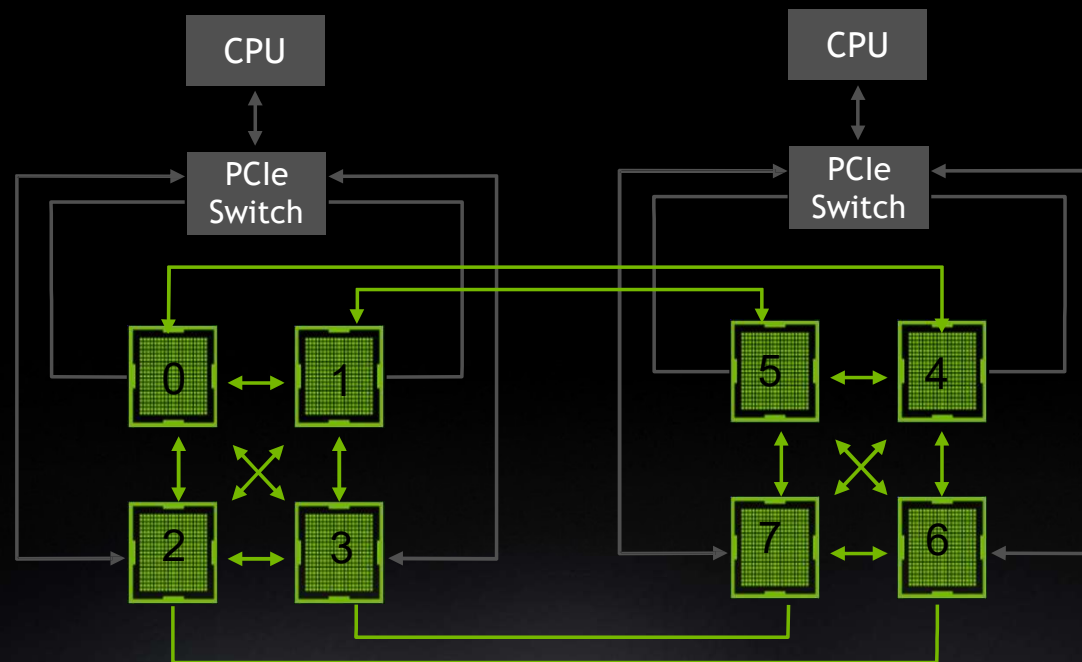
Data loading over
PCIe

DL DATA PARALLELISM - NVLINK



Gradient averaging over NVLink

DL DATA PARALLELISM - NVLINK



No sharing of communication resources: No congestion

NVIDIA Collective Communications Library (NCCL) 2

Multi-GPU and multi-node collective communication primitives

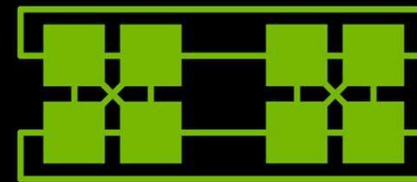
High-performance multi-GPU and multi-node collective communication primitives optimized for NVIDIA GPUs

Fast routines for multi-GPU multi-node acceleration that maximizes inter-GPU bandwidth utilization

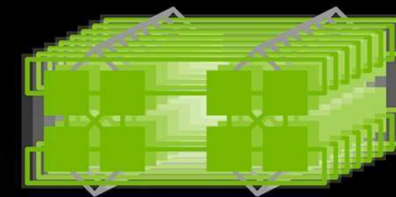
Easy to integrate and MPI compatible. Uses automatic topology detection to scale HPC and deep learning applications over PCIe and NVlink

Accelerates leading deep learning frameworks such as Caffe2, Microsoft Cognitive Toolkit, MXNet, PyTorch and more

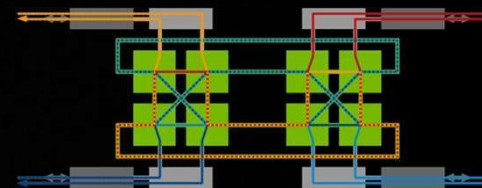
developer.nvidia.com/nccl



Multi-GPU:
NVLink
PCIe



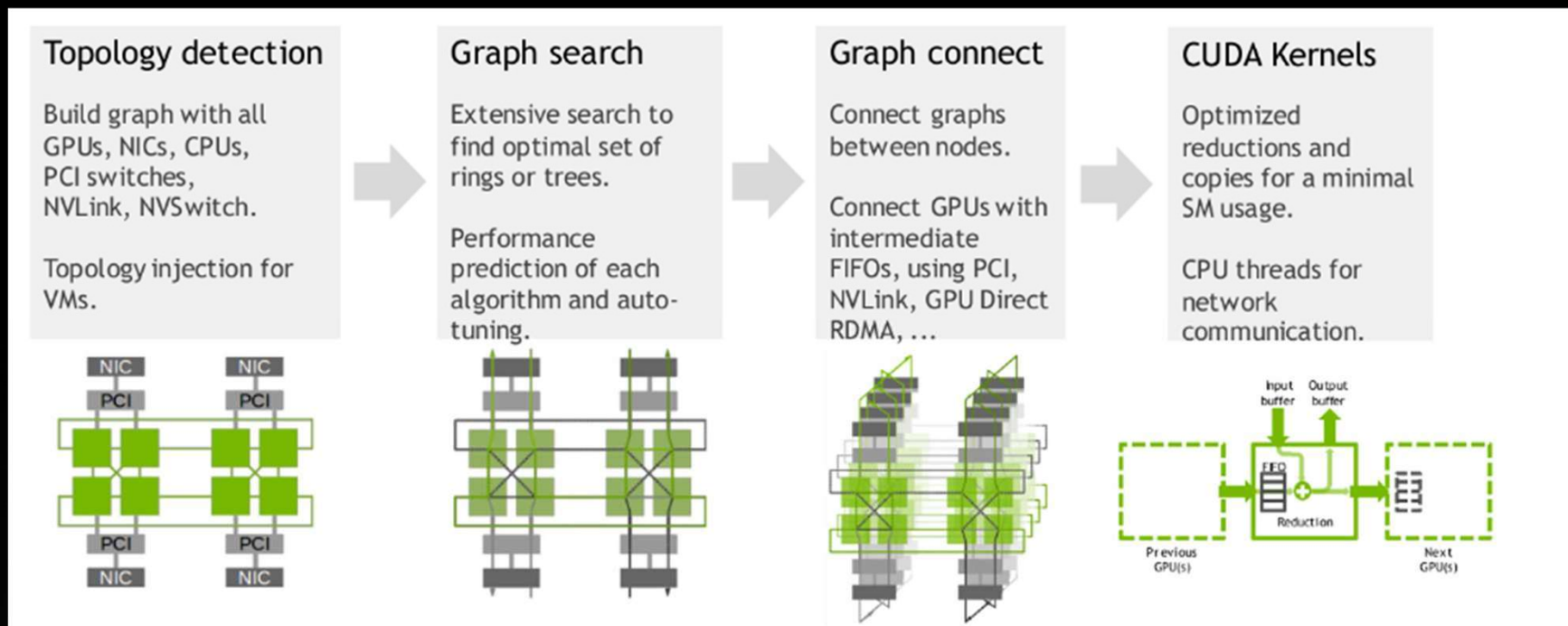
Multi-Node:
InfiniBand verbs
IP Sockets



Automatic
Topology
Detection

MULTI-NODE MULTI-GPU: NCCL

Topology-aware communication-centric NVIDIA library



NCCL
Architecture

MULTI-NODE MULTI-GPU: NCCL

API:

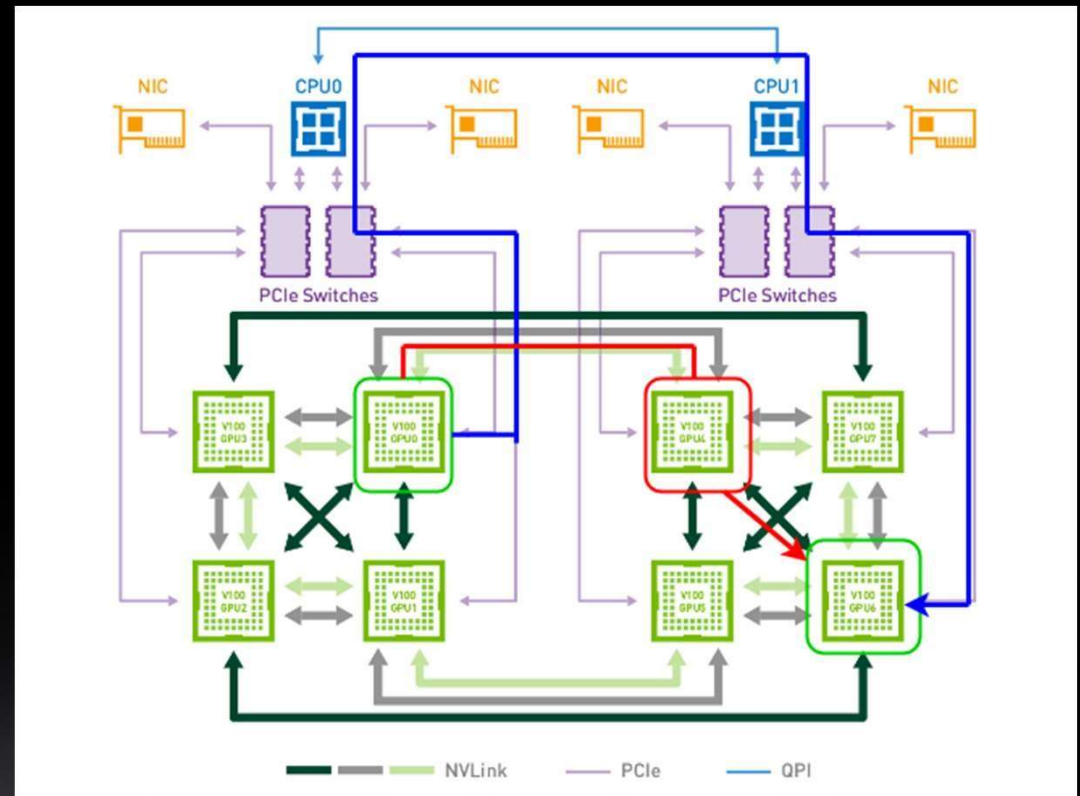
```
// Communicator creation
ncclGetUniqueId(ncclUniqueId* commId);
ncclCommInitRank(ncclComm_t* comm, int nranks, ncclUniqueId commId,
int rank);

// Communicator destruction
ncclCommDestroy(ncclComm_t comm);

// Point-to-point communication
ncclSend(void* sbuff, size_t count, ncclDataType_t type, int peer,
ncclComm_t comm, cudaStream_t stream);
ncclRecv(void* rbuff, size_t count, ncclDataType_t type, int peer,
ncclComm_t comm, cudaStream_t stream);

// Collective communication
ncclAllReduce(void* sbuff, void* rbuff, size_t count, ncclDataType_t type,
ncclRedOp_t op, ncclComm_t comm, cudaStream_t stream);
ncclBroadcast(void* sbuff, void* rbuff, size_t count, ncclDataType_t
type, int root, ncclComm_t comm, cudaStream_t stream);

// Aggregation/Composition
ncclGroupStart();
ncclGroupEnd();
```



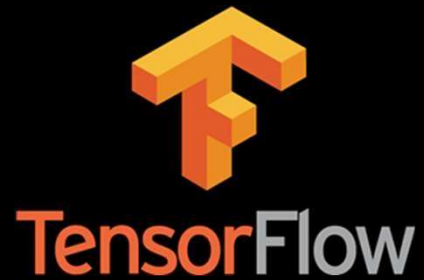
NCCL optimization: Blue path is taken by CUDA-aware MPI and Red path is taken by NCCL

A complex network diagram with numerous nodes (some green, some white) connected by thin lines, set against a dark background.

DISTRIBUTED DEEP LEARNING

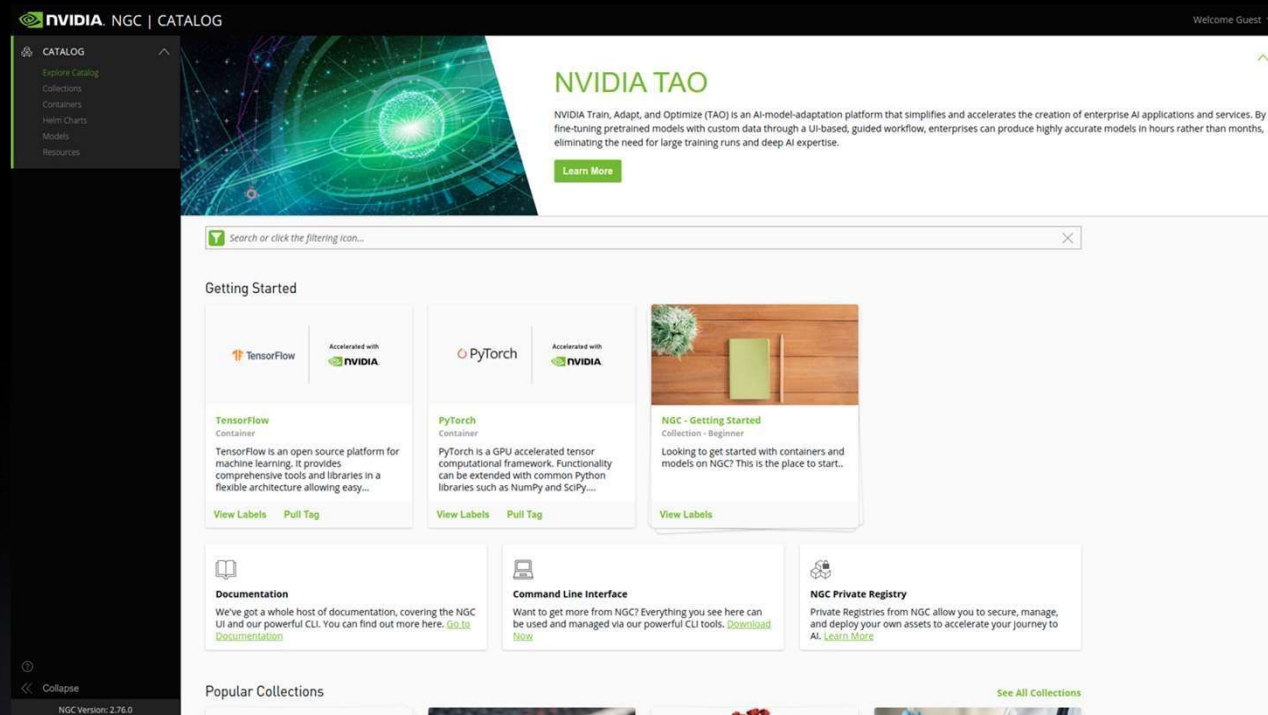
Session 2

FRAMEWORK SUPPORT



NVIDIA NGC SUPPORT

NGC Catalog contains containers that have Multi-node support built-in and can be readily deployed



The screenshot shows the NVIDIA NGC Catalog interface. The top navigation bar includes the NVIDIA logo, 'NGC | CATALOG', and a 'Welcome Guest' dropdown. A left sidebar contains links for 'Explore Catalog', 'Collections', 'Containers', 'Hello Charts', 'Models', and 'Resources'. The main content area features a large banner for 'NVIDIA TAO' (Train, Adapt, and Optimize) with a description and a 'Learn More' button. Below this is a search bar and a 'Getting Started' section with cards for TensorFlow, PyTorch, and 'NGC - Getting Started Collection - Beginner'. Further down are sections for 'Documentation', 'Command Line Interface', and 'NGC Private Registry'. At the bottom, there's a 'Popular Collections' section and a 'See All Collections' link.



INTRODUCTION TO TENSORFLOW DISTRIBUTED

TENSORFLOW

tf.distribute API

Tensorflow uses `tf.distribute.Strategy` API to distribute training across multiple GPUs, multiple machines or TPUs. Using this API, developer can distribute your existing models and training code with minimal code changes.

Tensorflow Strategies can be briefly summarized into two axes :

- **Synchronous vs Asynchronous** training
 - In sync training, all workers train over different slices of input data in sync and aggregate gradients at each step.
 - In async training, all workers are independently training over the input data and updating variables asynchronously.
- **Hardware platform**
 - You may want to scale your training onto multiple GPUs on one machine, or multiple machines in a network (with 0 or more GPUs each), or on Cloud TPUs.

TENSORFLOW

tf.distribute API

- **MirroredStrategy**

- tf.distribute.MirroredStrategy supports **synchronous** distributed training on multiple GPUs on one machine. It creates one **replica** per GPU device. Each variable in the model is **mirrored** across all the replicas. These variables are kept **in sync** with each other by applying identical updates.

- **MultiWorkerMirroredStrategy**

- tf.distribute.MultiWorkerMirroredStrategy is very similar to MirroredStrategy. It implements **synchronous** distributed training across **multiple workers**, each with potentially multiple GPUs. Similar to tf.distribute.MirroredStrategy, it creates copies of all variables in the model on each device across all workers.

- **CentralStorageStrategy**

- tf.distribute.experimental.CentralStorageStrategy does synchronous training as well. Variables are not mirrored, instead they are **placed on the CPU** and operations are replicated across all local GPUs. If there is only one GPU, all variables and operations will be placed on that GPU.

TENSORFLOW

`tf.distribute API`

- **ParameterServerStrategy**

- Parameter server training is a common data-parallel method to scale up model training on multiple machines. A parameter server training cluster consists of workers and parameter servers. Variables are created on parameter servers and they are read and updated by workers in each step.

- **OneDeviceStrategy**

- `tf.distribute.OneDeviceStrategy` is a strategy to place all variables and computation on a single specified device.

TENSORFLOW

Mirrored Strategy

1. Create an instance of the appropriate tf.distribute.Strategy

Ex : Creating a `MirroredStrategy` object. This will handle distribution, and provides a context manager to build your

model inside. (`tf.distribute.MirroredStrategy.scope`)

```
strategy = tf.distribute.MirroredStrategy()
```

2. Move the creation of model, optimizer and metrics inside strategy.scope()

```
with strategy.scope():  
    model = tf.keras.Sequential(...)  
    model.compile(...)
```

A complex network diagram with numerous nodes (white and yellow) connected by thin lines, set against a dark background.

INTRODUCTION TO HOROVOD

INTRODUCTION TO HOROVOD

- Horovod is a distributed deep learning training framework
- Supports TensorFlow, Keras, PyTorch, and Apache MXNet.
- Easy to Install and use
- Minimal code changes
- Proven Scalability
- Uses Advanced algorithms from HPC



horovod.ai

INTRODUCTION TO HOROVOD

Exascale Deep Learning

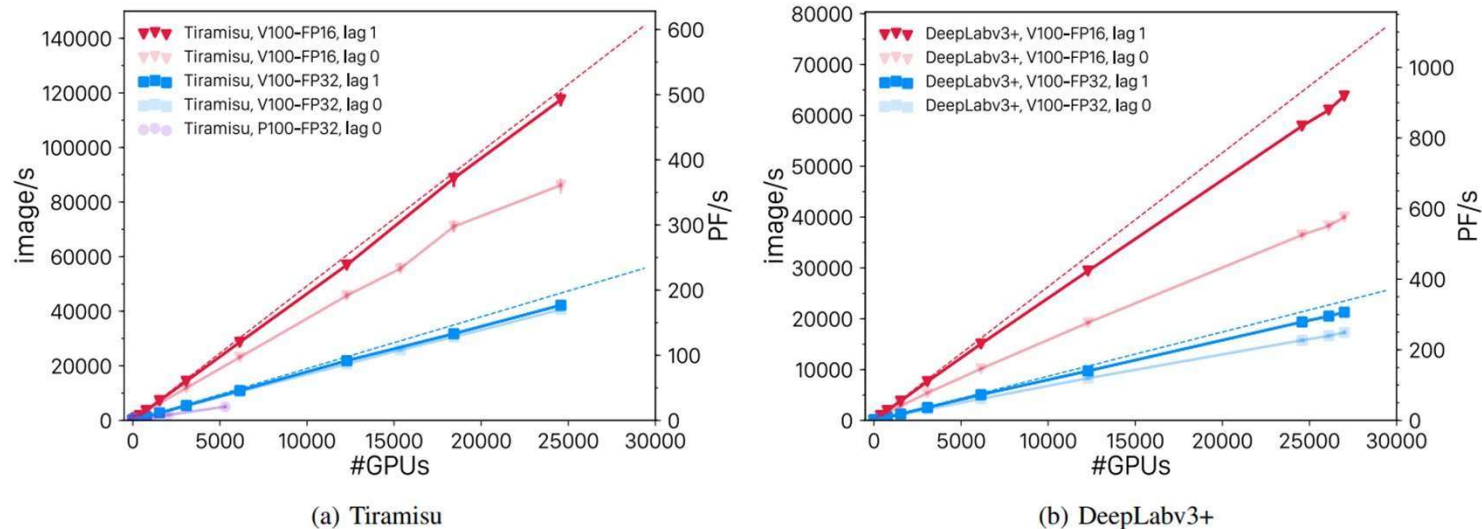


Fig. 4: Weak scaling results in terms of images/sec and sustained performance in PF/s on Summit (FP16 and FP32, Tiramisu and DeepLabv3+) and Piz Daint (FP32, Tiramisu). The dashed lines represent the ideal scaling lines for the different architectures and precisions.

HOROVOD

Horovod with Tensorflow

The following steps needs to be done while using Horovod with Tensorflow

1. Initialize Horovod
2. Pin each GPU to a single process.
3. Wrap the optimiser in Horovod Distributed optimiser. The distributed optimiser delegates gradient computation to the original optimiser, averages gradients using all-reduce or all-gather, and then applies those averaged gradients.
4. Broadcast the initial variable states from rank 0 to all other processes.
5. Modify your code to save checkpoints only on worker 0 to prevent other workers from corrupting them.

HOROVOD

Horovod with Tensorflow

- Initialise Horovod

```
hvd.init()
```

- Pin each GPU to a single process.

```
gpus = tf.config.experimental.list_physical_devices('GPU')
for gpu in gpus:
    tf.config.experimental.set_memory_growth(gpu, True)
if gpus:
    tf.config.experimental.set_visible_devices(gpus[hvd.local_rank()], 'GPU')
```

HOROVOD

Horovod with Tensorflow

- Wrap the optimiser in Horovod Distributed optimiser. The distributed optimiser delegates gradient computation to the original optimiser, averages gradients using all-reduce or all-gather, and then applies those averaged gradients.

```
opt = hvd.DistributedOptimizer(opt)
```

- Broadcast the initial variable states from rank 0 to all other processes.

```
hvd.callbacks.BroadcastGlobalVariablesCallback(0)
```

HOROVOD

Horovod with Tensorflow

- Modify your code to print stats and save checkpoints only on worker 0 to prevent other workers from corrupting them.

```
# Horovod: write logs only worker 0.  
verbose = 1 if hvd.rank() == 0 else 0  
model.fit(dataset, ..., verbose=verbose)
```

```
# Horovod: save checkpoints only on worker 0 to prevent other workers from corrupting them.  
if hvd.rank() == 0:  
    callbacks.append(keras.callbacks.ModelCheckpoint('./checkpoint-{epoch}.h5'))
```

HOROVOD

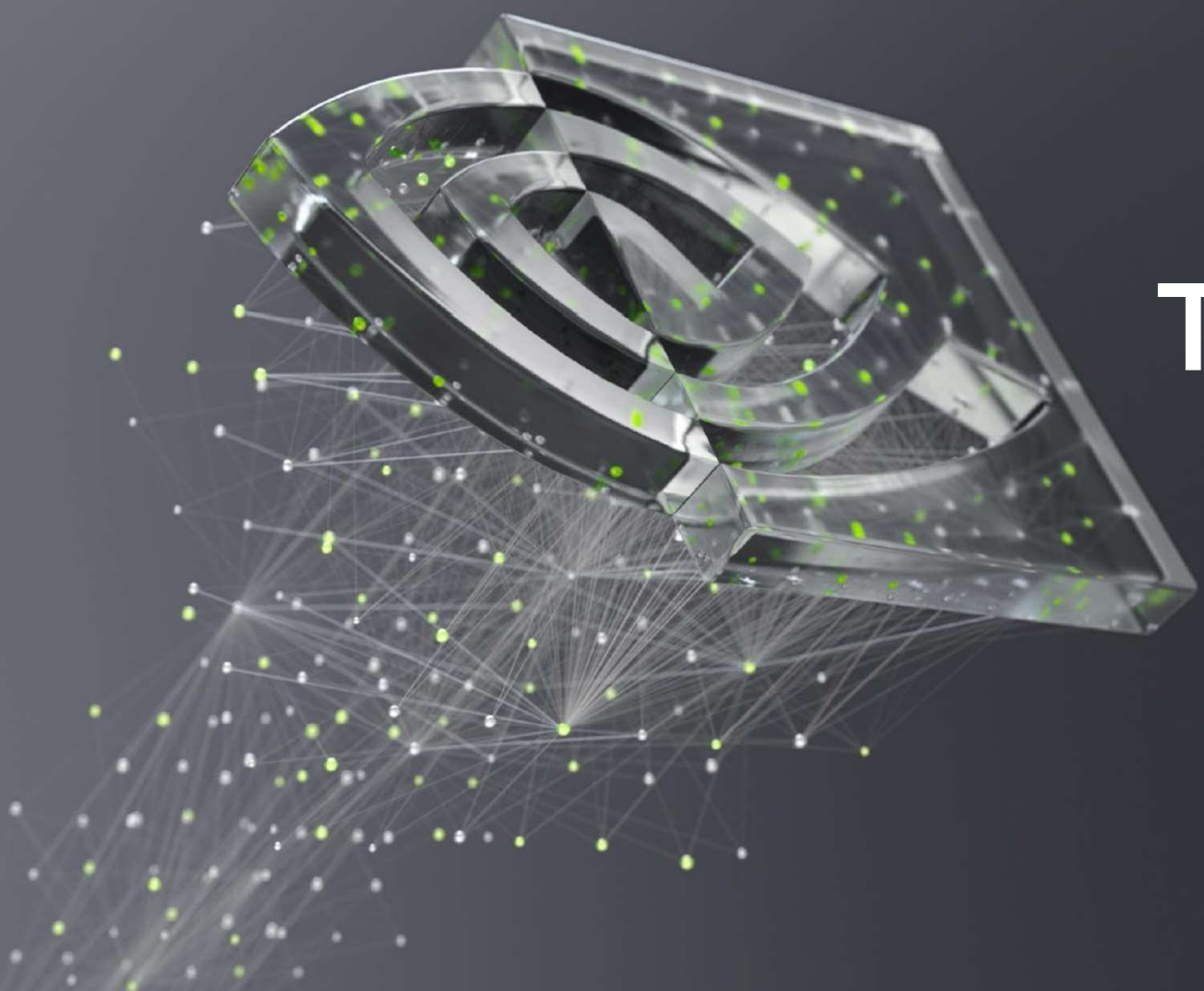
Running Horovod

Single Node

```
horovodrun -np 4 python train.py
```

Multi-node

```
horovodrun -np 8 -H server1:4,server2:4 python train.py
```



THANK YOU

